Assignment 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

   BFS assumes all edges have the exact same weight so it will ignore the edge cost and explores nodes level by level. Therefore, it finds path with fewest edges, not the shortest distance.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

   For BFS is turned into DFS, the first change is *state = frontier.popleft()* changes to *state = frontier.pop()*. The second change is *for neighbour, _ in graph.get(state,())* changes to *for neighbour, _ in reversed(graph.get(state, ()))*.

   The line that makes DFS order dependent is *for neighbour, _ in reversed(graph.get(state, ()))*.

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

   The first discovered path to the goal may not be the cheapest path. By waiting until the goal popped from the heap, UCS ensures the path to the goal that has been selected is the lowest cost path.

4. Lab1 reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

   expanded because it measures the amount of search work done by the algorithm. We don't use runtime_ms because it depends on the laptop speed, CPU and other hardware differences.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

   GBFS ignored *g(n)* which is the total cost from start to current node. This is why GBFS return non-optimal path in Lab 2 because it only considers heuristic *h(n)*. It may choose path that is closer to goal but has high total cost.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

   UCS expands Y first because Y has lower *g* value than X.
   GBFS expands X first because X has lower *h* value than Y.
   A* expands Y first because Y has lower *(g+h)* value than X.

7. Why must *h* be expressed in the same units as the edge costs?

   This is because A* adds *g(n)* and *h(n)*. Since edge cost is in kilometres, the heuristic should be in kilometres too.

8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

   This is because Dijkstra can start from the goal and calculate the shortest cost from every airport to the goal. If the graph is not reversed, it will find the shortest paths starting from goal rather than shortest paths leading to the goal.

9. Weighted A⋆ with $w = 1.5$ changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

   The path SIN→EZE remained unchanged because the optimal route was still preferred even when heuristic was weighted by 1.5. The other route had worse weighted values, so weighted A* continue to select the same route.